

RelUI: Relational Behavioral Consistency Testing for Web UIs

CS 527 Research Project Report: Group 2

Ajmal Ali

Muneeb Hoda

University of Illinois at Urbana-Champaign

Urbana, Illinois, USA

{aali274,shoda6}@illinois.edu

Abstract

Modern Web UI bugs often hide between pages rather than inside one page. A confirmation screen can look normal while losing data after back navigation; a protected dashboard can render correctly while being reachable without login. This paper presents RelUI, a TypeScript and Playwright prototype for relational behavioral consistency testing of Web UIs. RelUI explores browser traces, canonicalizes states, infers a state-transition graph, and checks explicit workflow invariants over traces and endpoints. We evaluate the final artifact on four benchmark workflows, 20 subject variants, and 16 injected faults. The final run explored 454 replayable traces, discovered 230 canonical states and 467 transitions, detected all 16 injected faults, and reported zero clean false positives in 250.08 seconds. Lightweight structural and visual baselines detected only 5 and 0 faults, respectively, showing that workflow correctness needs relational oracles beyond local DOM or screenshot checks.

1 Introduction

Web applications increasingly encode critical behavior as multi-step workflows: login, checkout, account settings, and money transfer flows all depend on action history. These workflows fail in ways that are hard to detect with one-state checks. A page can contain the right controls and still violate an authentication precondition; two valid paths can reach the same confirmation URL but disagree about preserved user data.

RelUI targets this class of bug. It treats a Web UI as an explored transition system and checks relational invariants over traces, graph edges, and endpoint states. The final artifact is implemented in TypeScript on top of Playwright [3]; it emits JSON, CSV, HTML, screenshots, and LaTeX-ready tables.

Figure 1 summarizes the final run. The project makes three contributions: (1) a relational approach for Web UI workflow testing, (2) a runnable Playwright-based tool with replayable counterexamples, and (3) an evaluation over four workflows showing that relational oracles detect faults missed by local structural and visual baselines.

2 Approach Overview

RelUI receives a subject URL, input profiles, seed traces, exploration bounds, and invariant definitions. It drives the UI in a real browser, snapshots reached states, merges equivalent states by semantic fingerprint, builds a labeled graph, and checks five invariant families.

The main design decision is to make the oracle relational. Instead of asking only whether a page looks different, RelUI asks whether a state is justified by the trace that reached it and whether alternative traces that implement the same task converge to equivalent endpoints.

4 apps	20 variants	16/16 faults	0 clean FPs
454 traces	230 states	467 edges	250.08s runtime

Figure 1: Final run scorecard generated from the upgraded artifact.



Figure 2: RelUI pipeline: browser exploration, semantic canonicalization, graph inference, relational checking, and evidence export.

3 Approach Details

3.1 Exploration and Canonicalization

RelUI combines deterministic seed traces with bounded action discovery. Seed traces guarantee coverage of security- and validation-relevant paths, while action discovery explores visible anchors, buttons, submit inputs, and role-button elements. Form actions are filled from named profiles such as valid, admin, and invalid.

Each reached page is snapshotted as URL, title, visible text, visible controls, form values, and screenshot path. RelUI normalizes URLs, text, and control signatures, then hashes the semantic representation. Nodes with the same fingerprint are merged into a canonical state.

3.2 Relational Oracles

The final tool implements five invariant families: authentication/precondition, validation-blocking, action-forbidden-state, forbidden-transition, and path-equivalence. Listing 1 shows the new Transfer benchmark’s path-equivalence invariant. It checks that a normal transfer and a back/edit transfer path reach equivalent receipts after confirmation.

Listing 1: Transfer path-equivalence invariant from the final configuration.

```
1 {
2   id: "transfer-confirmation-paths-equivalent",
3   type: "path-equivalence",
4   severity: "medium",
5   description:
6     "Normal and back/edit transfer paths must converge to the same receipt state",
7   endpoint: { textIncludes: "Transfer Complete" },
```

Algorithm 1: RelUI exploration and checking

Input: Subject S , seed traces T_0 , profiles P , invariants I , bounds

Output: State graph G and violations V

```

1  $Q \leftarrow$  empty trace plus  $T_0$ ;  $G, V \leftarrow \emptyset$ ;
2 while  $Q$  has traces and trace bound is not exceeded do
3   pop trace  $t$ ; reset browser storage; navigate to  $S.url$ ;
4   replay  $t$  with Playwright and fill forms from  $P$ ;
5   snapshot reached pages and canonicalize them into state
   nodes;
6   add labeled transitions between consecutive canonical
   states;
7   if  $t$  is below depth bound then
8     discover visible enabled actions and enqueue unseen
     trace extensions;
9 foreach invariant  $i \in I$  do
10  evaluate  $i$  over traces, states, and graph edges; add
    counterexamples to  $V$ 
11 return  $G, V$ ;
```

```

8 candidateActionSelectors: ['[data-testid="confirm-transfer"]'],
9 profiles: ["valid"],
10 minDistinctPaths: 2
11 }
```

3.3 Evidence and Exports

Every violation records invariant ID, type, severity, trace IDs, state IDs, replay steps, screenshots, and text deltas. The final exporter also computes app-level coverage and invariant-family coverage. This lets the report separate two questions: whether faults are detected, and what exploration/model size was needed to detect them.

4 Experimental Evaluation

4.1 Evaluation Plan

We evaluate three research questions:

- RQ1:** How effectively does RelUI detect injected workflow-level UI faults?
- RQ2:** Which faults are missed by structural and visual baselines?
- RQ3:** What exploration cost and model size does RelUI incur across benchmark apps?

The final benchmark suite contains Auth, Checkout, Settings, and Transfer workflows. Each workflow has one clean variant and four faulty variants, yielding 20 variants and 16 injected faults. The baselines are intentionally lightweight: a structural baseline flags unseen visible-control signatures, and a visual baseline compares screenshots from matching traces with pixel difference. They provide a diagnostic contrast, not a claim against full industrial testing suites.

4.2 Evaluation Results

4.2.1 RQ1: Fault Detection. RelUI detected all 16 injected faults and produced zero relational violations on the four clean variants.

Table 1: App-level coverage and cost from the final run.

App	Variants	Faults	Detected	States	Trans.	Traces	Actions	Avg Len.	Max Len.	Runtime (s)
Auth	5	4	4/4	49	95	90	10	1.73	4	45.92
Checkout	5	4	4/4	69	150	142	11	2.25	6	91.77
Settings	5	4	4/4	45	95	100	9	1.2	3	40.29
Transfer	5	4	4/4	67	127	122	9	2.03	5	72.09

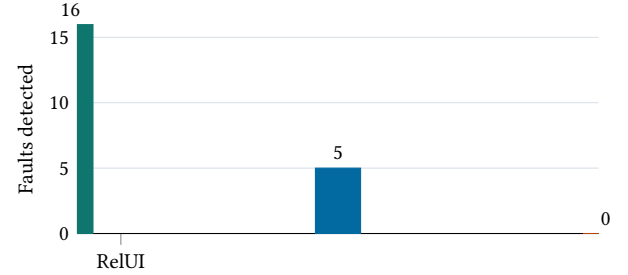


Figure 3: Fault detection comparison. RelUI detects all 16 injected faults; structural and visual baselines detect 5 and 0.

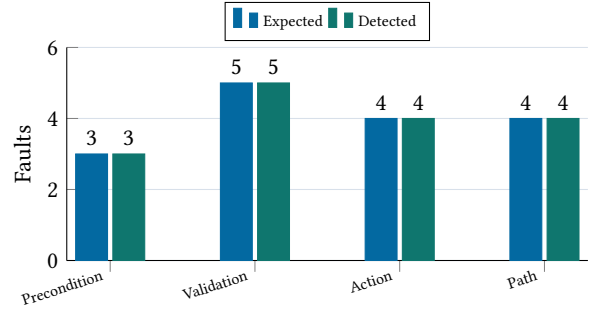


Figure 4: Invariant-family contribution. Each configured or-acle family detects all faults assigned to it.

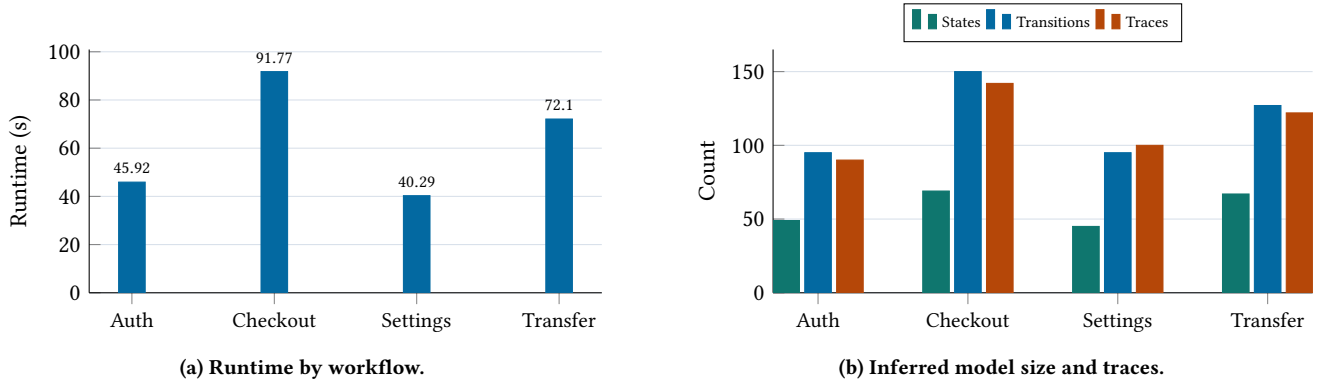
The complete run discovered 230 canonical states, 467 transitions, and 454 replayable traces. Table 1 shows that Checkout remained the largest workflow by states, transitions, traces, and runtime, while Transfer added a second multi-step, data-preservation-heavy benchmark.

4.2.2 RQ2: Baseline Misses. Figure 3 shows the headline comparison. Structural checking detects five faults that change visible control signatures: one Auth bypass, two Checkout validation failures, and two Transfer validation failures. It misses logout retention, role escalation, payment bypass, cancel/reset persistence, stale quick-save paths, and path-equivalence faults because those failures are semantically wrong but structurally ordinary. The visual baseline detects none under the configured threshold because most faulty pages are intentionally plausible.

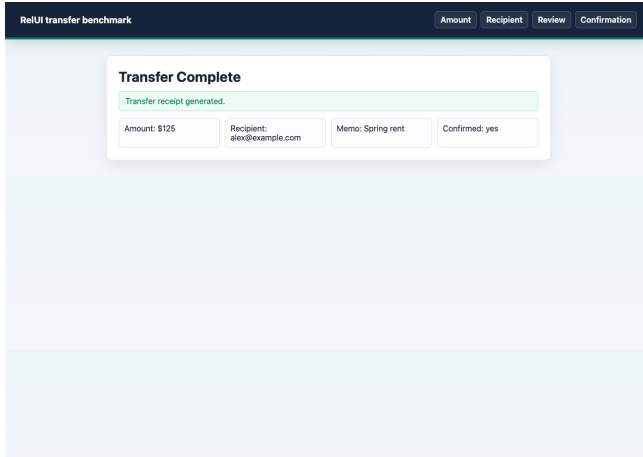
4.2.3 RQ3: Exploration Cost. Figure 5 groups runtime and model size by workflow. Checkout is slowest at 91.77 seconds because it has the deepest workflow and largest transition set. Transfer adds 122 traces and 127 transitions in 72.10 seconds, showing that the added benchmark increases evaluation breadth without dominating total cost. Settings is smallest because most traces are short profile or notification actions.

Table 2: Fault coverage matrix. Structural and visual columns show whether the lightweight baselines detected the same faulty variant.

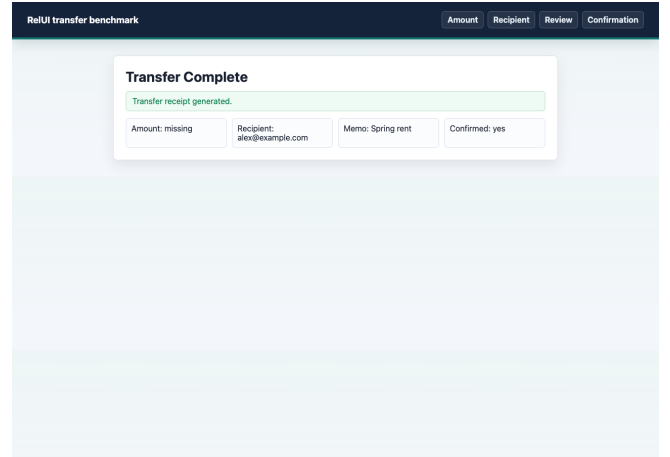
Fault	Workflow defect	ReUI oracle family	Structural	Visual
F-AUTH-1	Dashboard bypasses authentication	auth-precondition	Yes	–
F-AUTH-2	Logout retains protected access	action-forbidden-state	–	–
F-AUTH-3	User role reaches admin console	auth-precondition	–	–
F-AUTH-4	Invalid login reaches dashboard	validation-blocking	–	–
F-CHECKOUT-1	Invalid contact advances to payment	validation-blocking	Yes	–
F-CHECKOUT-2	Express review bypasses payment	action-forbidden-state	–	–
F-CHECKOUT-3	Negative quantity advances checkout	validation-blocking	Yes	–
F-CHECKOUT-4	Back path loses contact details	path-equivalence	–	–
F-SETTINGS-1	Quick profile save writes stale state	path-equivalence	–	–
F-SETTINGS-2	Cancel persists edited display name	action-forbidden-state	–	–
F-SETTINGS-3	Reset leaves edited display name	action-forbidden-state	–	–
F-SETTINGS-4	Quick notification save discards state	path-equivalence	–	–
F-TRANSFER-1	Invalid amount advances to recipient	validation-blocking	Yes	–
F-TRANSFER-2	Invalid recipient advances to review	validation-blocking	Yes	–
F-TRANSFER-3	Receipt reachable without confirm action	auth-precondition	–	–
F-TRANSFER-4	Back/edit path loses transfer amount	path-equivalence	–	–


Figure 5: Runtime and model size after adding Transfer. The cost tracks workflow depth and branching more than the number of variants alone.

4.2.4 Representative Counterexample. Figure 6 shows the new Transfer path-equivalence fault. A normal transfer receipt preserves \$125; the back/edit path reaches the same receipt state class after confirmation but loses the amount. The page remains visually plausible, which explains why local visual comparison is weak here.



(a) Normal path: confirmed receipt keeps the amount.



(b) Back/edit path: receipt loses the amount.

Figure 6: Transfer counterexample for F-TRANSFER-4. The two traces both end after confirm-transfer, but endpoint fingerprints differ: one receipt reports Amount: \$125, while the other reports Amount: missing.

5 Related Work

Model-based testing. Model-based testing derives tests from behavioral models [5]. RelUI also uses a transition model, but infers it from browser traces and uses it mainly for relational oracle checking rather than for exhaustive model-derived generation.

Web state crawling and invariants. Crawljax explores Ajax applications and infers state-flow graphs [1]. Invariant-based Web testing checks discovered states against state and DOM constraints [2]. RelUI builds on these ideas but checks relationships between traces: required actions before protected endpoints, invalid profiles before forbidden states, and equivalent paths to the same task endpoint.

GUI testing frameworks. GUITAR provides an extensible GUI testing framework across workflows and platforms [4]. RelUI is narrower and Web-focused, but produces compact Playwright replay evidence around relational workflow defects.

Browser automation and LVLM GUI testing. Playwright supplies reliable browser automation and screenshots [3]. LVLM-based tools such as VETL use visual understanding for GUI exploration and input generation [6]. RelUI is complementary: its oracle is explicit and auditable rather than learned.

6 Threats to Validity

Internal validity. The faults are controlled injections, and each expected fault is mapped to an invariant in the configuration. This improves auditability but means results depend on the correctness of those mappings. Clean variants reduce false-positive risk, but they do not prove the invariants are complete.

External validity. The four workflows are realistic surrogates, not production systems. They cover authentication, checkout, settings, and transfer behavior, but not all asynchronous, third-party, or highly dynamic UI patterns.

Construct validity. The structural and visual baselines are lightweight diagnostic baselines. Their role is to show that local

structure and appearance miss relational defects, not to replace a full comparison with mature commercial suites.

Reliability. RelUI resets browser storage before traces and uses deterministic input profiles. Remaining nondeterminism can come from browser timing and screenshots. The artifact preserves graphs, traces, screenshots, CSV files, and HTML evidence to support reruns and manual inspection.

7 Conclusion

RelUI shows that Web UI correctness often depends on relationships across actions and paths. The final artifact expands the evaluation to four workflows, 20 variants, and 16 injected faults; it detects all faults with zero clean false positives while local structural and visual baselines detect only 5 and 0. The results support the core claim: workflow-level UI testing needs explicit relational oracles, not only page-local checks.

References

- [1] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. 2012. Crawling Ajax-Based Web Applications Through Dynamic Analysis of User Interface State Changes. *ACM Transactions on the Web* 6, 1 (2012), 1–30. doi:10.1145/2109205.2109208
- [2] Ali Mesbah, Arie van Deursen, and Danny Roest. 2012. Invariant-Based Automated Testing of Modern Web Applications. *IEEE Transactions on Software Engineering* 38, 1 (2012), 35–53. doi:10.1109/TSE.2011.28
- [3] Microsoft Playwright. 2026. Playwright Browser Testing Documentation. <https://playwright.dev/docs/browsers>. Accessed 2026-05-05.
- [4] Bao N. Nguyen, Bryan Robbins, Ishan Banerjee, and Atif Memon. 2014. GUITAR: An Innovative Tool for Automated Testing of GUI-Driven Software. *Automated Software Engineering* 21, 1 (2014), 65–105. doi:10.1007/s10515-013-0128-9
- [5] Mark Utting, Alexander Pretschner, and Bruno Legeard. 2012. A Taxonomy of Model-Based Testing Approaches. *Software Testing, Verification and Reliability* 22, 5 (2012), 297–312. doi:10.1002/stvr.456
- [6] Siyi Wang, Sinan Wang, Yujia Fan, Xiaolei Li, and Yepang Liu. 2024. Leveraging Large Vision Language Model for Better Automatic Web GUI Testing. doi:10.48550/arXiv.2410.12157 arXiv:2410.12157 [cs.SE]